COLLECTION TOTAL — RÉCONCILIATION

# L1 — L'Origine

## Partie A — Le Premier Signal

---

**ToTAL — Réconciliation**

Collection en 21 livres — 14 volumes Réconciliation & 7 Guides

◆ SCEAU ToTAL ◆

Espace réservé — logo officiel

---

*Images et couverture par Soléane Bélanger*

---

### Crédits visuels

*Les images et la couverture de ce livre sont des chefs-d'œuvre réalisés par Soléane Bélanger.*

---

ToTAL — Réconciliation est une collection originale. Tout droit réservé. Toute reproduction, même partielle, est interdite sans autorisation explicite. Les personnages, organisations et situations décrits dans les scènes fictionnalisées sont des composites inspirés de l'expérience de terrain ; toute ressemblance avec des personnes ou organisations réelles est fortuite.

---

AVANT DE COMMENCER

# Repères de lecture ToTAL

**Comment lire ce livre**

Ce livre peut se lire seul ou en équipe. Lorsque vous le lisez seul, laissez les questions de réflexion vous accompagner comme un interlocuteur silencieux. Prenez le temps de vous arrêter aux encadrés de fin de chapitre — ils ne sont pas décoratifs. Ils sont les points de pivot.

Lorsque vous le lisez en équipe, utilisez les questions de réflexion comme point de départ de vos échanges. Il n'y a pas de bonne réponse. Il y a des conversations honnêtes, et des conversations évitées. ToTAL préfère les premières.

## Convention linguistique : français et anglais

La collection ToTAL est rédigée en français. Certains termes issus de l'anglais sont tellement présents dans la pratique quotidienne des équipes logicielles qu'en les traduisant sans les identifier, nous risquerions de créer de la confusion. Nous avons choisi une approche simple : le terme français est utilisé en premier, et son équivalent anglais courant figure entre parenthèses à la première occurrence dans chaque livre.

| Terme français (utilisé dans le texte) | Équivalent anglais courant |
|---|---|
| Carnet de produit | backlog |
| Chaîne de livraison | pipeline |
| Itération | sprint |
| Terminé | done |
| Livraison | release |
| Soutien | support |
| Tableau de bord | dashboard |
| Billet de travail | ticket |
| Infonuagique | cloud |
| Compilation | build |

### Posture non-dogmatique

ToTAL n'est pas une religion. Il ne prétend pas que son approche est la seule valable, ni que les autres méthodes, cadres ou pratiques que vous utilisez sont erronés. ToTAL est une lentille. Il cherche à poser des questions avant de proposer des réponses, à nommer ce qui est flou avant de proposer un outil pour le structurer, à respecter l'intelligence des équipes qui vivent déjà avec la complexité au quotidien.

Si vous trouvez que quelque chose dans ce livre contredit ce que vous savez de votre réalité, la posture ToTAL recommande de ne pas ignorer cet inconfort. Il est peut-être porteur d'un signal important.

---

NAVIGATION

# Table des matières


### Page de titre & Crédits

**Repères de lecture ToTAL**

**Avant-propos — Ce que ce livre est et ce qu'il n'est pas**

**Prologue personnel — Le premier bug que personne ne voulait croire**

*Le bureau du père*

*L'enfant qui chicane la machine*

*Un archétype de tous les signaux ignorés*

*De l'enfant au praticien : la continuité d'un regard*

**Prologue — La question qui a tout déclenché**

*L'Eureka ToTAL*

*Les symptômes du désordre ordinaire*

*La révélation : les concepts eux-mêmes*

*Encadrés de fin de prologue*

**Introduction — Pourquoi rien ne suffit, et pourquoi cela s'aggrave malgré tout**

*Les organisations logicielles modernes*

*Trois profils organisationnels*

*ORIGINALIUM, DREAM-TEAM et BEASTEST*

*La chaîne fondatrice*

*L'intelligence artificielle comme révélateur*

*Encadrés de fin d'introduction*

**Partie 1 — Le chaos que les organisations ont appris à normaliser**

*Note de partie*

Chapitre 1 — Le paradoxe des organisations modernes

*Avoir tout, manquer de maîtrise*

*Scène ORIGINALIUM : La réunion de direction*

*Confondre activité et maîtrise*

*La première question ToTAL*

*Scène ORIGINALIUM : La réunion produit floue*

*Encadrés de fin de chapitre*

Chapitre 2 — Accélérer, stagner ou décélérer

*Les trois profils en profondeur*

*ORIGINALIUM : les trois simultanément*

*Le faux remède : ajouter des ressources*

*Carnet de terrain de Karim (QA)*

*La vraie question*

*Encadrés de fin de chapitre*

Chapitre 3 — Le chaos n'est pas seulement de la mauvaise volonté

*La tentation du coupable*

*Le chaos comme symptôme de système*

*Le blâme ferme les boucles d'apprentissage*

*Carnet de terrain — expérience de terrain*

*La qualité du système humain*

*Encadrés de fin de chapitre*

**Quatrième de couverture (Partie A)**

---

AVANT-PROPOS

# Ce que ce livre estet ce qu'il n'est pas

> *L'Origine est le livre fondateur de la collection ToTAL. Il ne tente pas de tout couvrir. Il plante une graine.*

Vous tenez entre les mains le premier livre d'une collection de vingt-et-un. Ce n'est pas un manuel. Ce n'est pas un guide de démarrage rapide. Ce n'est pas un recueil de bonnes pratiques triées pour impressionner.

Ce livre est une invitation à regarder avant d'agir.

Il pose une question, situe cette question dans une scène fondatrice, reconnaît un chaos que beaucoup d'équipes vivent déjà sans toujours l'avoir nommé, et propose de voir autrement. Pas de voir mieux que les autres — de voir ce que les outils et les processus, seuls, ne peuvent pas montrer.

Les vingt livres qui suivent portent chacun leur part du système ToTAL. Ils entrent dans le détail : les pratiques, les techniques, les outils, les patterns récurrents, les pièges, les leviers. Ensemble, ils forment une discipline complète.

Mais L'Origine est différent. Il est le pourquoi avant le comment. Il est la conviction avant la méthode. Il est la cicatrice avant la leçon.

Si vous cherchez des réponses immédiates, certains des livres suivants répondront plus directement à vos besoins. Mais si vous cherchez à comprendre pourquoi les réponses immédiates finissent souvent par créer de nouveaux problèmes — alors commencez ici.

Ce livre n'est pas exhaustif. Cette limite est intentionnelle. Il y a des choses que l'on ne peut apprendre qu'en commençant, en se trompant un peu, en reconnaissant le signal. L'Origine existe pour créer cet espace : un espace où quelque chose peut commencer à devenir clair avant même que le travail ne commence vraiment.

> *Vingt livres porteront le comment. Ce livre-ci porte le pourquoi. Il n'est pas moins rigoureux pour autant. Il est simplement honnête sur ce qu'il est.*

PROLOGUE PERSONNEL

# Le premier bug que personnene voulait croire

> *Il y a une scène dans chaque vie professionnelle qui contient, à l'état brut, tout ce que l'on passera des années à comprendre. Voici la mienne.*

## Le texte fondateur

Très jeune, je me glissais dans le bureau de mon père pour faire de l'ordinateur. Je n'avais pas encore le vocabulaire pour expliquer ce que je faisais. Je fouinais. J'essayais. Je cliquais. Je testais des choses improbables. Je découvrais les premières formes d'Internet comme on découvre un territoire immense, encore mal cartographié, où chaque écran pouvait cacher une surprise.

Mais ce dont je me souviens le plus, ce n'est pas l'émerveillement. C'est la frustration. Je chicanais l'ordinateur. Je lui en voulais de faire des choses incohérentes, d'afficher des comportements qui ne tenaient pas debout, de produire des erreurs que je sentais réelles sans pouvoir les nommer. Mon père trouvait cela drôle. Pour lui, l'ordinateur ne pouvait pas se tromper. S'il y avait une erreur, elle venait forcément de moi.

Pendant des années, cette scène est devenue une blague : l'enfant qui accusait la machine. L'enfant qui prétendait voir des erreurs dans un système supposé parfait. Avec le recul, je peux sourire. Mais je sais aussi autre chose : je n'étais pas fou. Il y avait réellement des bugs. Je n'avais ni l'autorité, ni l'expérience, ni les bons mots, mais j'avais déjà rencontré une vérité qui allait me suivre toute ma vie professionnelle : un problème réel peut rester invisible lorsque ceux qui regardent le système croient déjà qu'il ne peut pas se tromper.

Mon père avait raison sur une chose : l'ordinateur ne décidait pas, par lui-même, de faire une erreur. Mais il avait tort sur une autre : une machine peut exposer les erreurs d'un système humain. Elle peut révéler une logique incomplète, une conception fragile, une programmation imparfaite, une configuration mal comprise ou une attente mal définie.

Cette première injustice — voir quelque chose de réel sans être cru — a laissé une trace. Plus tard, je l'ai revue partout sous d'autres formes. Des utilisateurs voyaient des irritants que le système appelait des exceptions. Des personnes qualité voyaient des risques que les projets traitaient comme des détails. Des équipes voyaient le flou avant qu'il devienne une crise, mais on leur demandait d'avancer quand même.

Aujourd'hui, nous répétons parfois avec l'intelligence artificielle ce que certains croyaient hier avec l'ordinateur : la machine impressionne, donc elle doit avoir raison. Mais l'IA, comme l'ordinateur de mon enfance, amplifie surtout la qualité du système qui la nourrit. Si le besoin est flou, elle peut générer du flou avec élégance. Si les critères sont faibles, elle peut produire une réponse convaincante mais fragile. Si la décision est mal comprise, elle peut accélérer une mauvaise direction.

ToTAL vient en partie de cette scène fondatrice : avant le code, avant le test, avant l'outil, avant l'automatisation et avant l'IA, il y a quelqu'un qui ose dire :

quelque chose n'est pas clair, quelque chose ne tient pas, quelque chose mérite d'être regardé avant d'être amplifié.

---

## Un archétype de tous les signaux ignorés

Cette scène du bureau paternel n'est pas une anecdote nostalgique. Elle est, avec le recul de plusieurs décennies de pratique dans le monde du logiciel, un archétype remarquablement fidèle de ce que vivent des dizaines d'équipes chaque semaine dans des organisations de toutes tailles.

Pensez-y : un observateur perçoit quelque chose de dysfonctionnel. Il ne dispose pas encore du vocabulaire institutionnel pour le nommer. Il ne détient pas l'autorité formelle pour l'inscrire à l'ordre du jour. Et ceux qui ont cette autorité — ceux qui possèdent le contexte, l'expérience, la légitimité — ont déjà une conviction sur le système : il ne peut pas se tromper. Donc l'erreur est ailleurs. L'erreur est chez celui qui signale.

Ce mécanisme, nous l'avons tous vécu sous une forme ou une autre. L'utilisateur qui rapporte un comportement étrange et qu'on renvoie vers sa propre erreur de manipulation. La personne d'assurance qualité (AQ) qui identifie un risque dans les exigences et qu'on rassure d'un « ça va se clarifier plus tard ». Le développeur junior qui voit un flou dans l'architecture et qui n'ose pas le dire parce que l'architecte senior a l'air si confiant. Le chef de produit qui sent que les métriques ne racontent pas toute l'histoire, mais qui n'a pas de langage pour contester le tableau de bord (dashboard) que la direction regarde avec satisfaction.

Le signal est là. Le signal est réel. Mais le signal reste invisible parce que le système — le système humain, organisationnel, culturel — n'est pas encore prêt à le recevoir.

C'est cette réalité que ToTAL tente de transformer. Non pas en ajoutant un autre outil. Non pas en prescrivant un nouveau processus. Mais en créant les conditions pour que les signaux puissent être nommés, entendus, et convertis en apprentissage avant de devenir des crises.

## L'enfant qui chicane la machine deviendra le praticien

Il y a quelque chose de profondément formateur dans le fait de sentir un problème que l'on ne peut pas encore articuler. Cette expérience — sentir avant de savoir — est souvent décrite comme une forme d'intuition. Mais c'est aussi, plus précisément, la naissance d'une sensibilité systémique.

L'enfant dans le bureau ne sait pas encore ce qu'est un bug. Il ne connaît pas la différence entre un problème de logiciel et un problème de configuration. Il ignore la distinction entre un comportement non documenté et un vrai défaut. Et pourtant, il sent que quelque chose ne tient pas. Il sent la dissonance entre ce qu'il attendait et ce qui se produit. Et il refuse — instinctivement — d'accepter que cette dissonance soit son propre problème.

Cette résistance est précieuse. Dans le monde du logiciel, la tendance la plus dangereuse n'est pas de voir des problèmes qui n'existent pas. La tendance la plus dangereuse est d'arrêter de voir les problèmes qui existent parce que le système autour de nous nous dit qu'ils ne devraient pas exister.

Des années plus tard, ce même enfant — devenu praticien — a appris le vocabulaire. Il a appris à distinguer un risque d'une incertitude, un défaut d'une décision de conception, un symptôme d'une cause. Il a appris à transformer ce qu'il sentait en quelque chose qui pouvait être lu, mesuré, et transmis. Mais le moteur est resté le même : une intolérance douce au flou qui passe inaperçu.

ToTAL est, dans un sens très direct, la cristallisation de cette intolérance en une discipline. Une discipline qui dit : le flou n'est pas un détail administratif.

Le flou non résolu est la source principale des reprises, des conflits, des livraisons qui décevra et des transformations qui n'atterrissent pas.

## Le lien entre la scène d'enfance et la naissance de ToTAL

Le lien entre ce bureau du père et la naissance de ToTAL est direct. Il n'est pas métaphorique. Il est causal.

Parce que cette expérience fondatrice — voir quelque chose de réel sans être cru — a créé une sensibilité permanente à ce qui passe sous le radar dans les organisations logicielles. Elle a créé une attention particulière aux moments où quelqu'un signale quelque chose que le système n'est pas prêt à entendre. Elle a créé une méfiance durable envers les tableaux de bord rassurants, les rituels bien tenus, et les métriques vertes qui cachent une maîtrise incertaine.

Et elle a créé, à terme, la conviction que le problème n'est pas les individus. Le problème est le système qui rend certains signaux inaudibles.

ToTAL est né pour rendre ces signaux audibles. Pour donner aux équipes non seulement les outils, mais aussi — et surtout — le langage et la posture pour dire : quelque chose n'est pas clair ici. Quelque chose mérite d'être regardé avant d'être automatisé. Quelque chose mérite d'être compris avant d'être livré.

> *Avant le code, avant le test, avant l'outil, avant l'automatisation et avant l'IA, il y a quelqu'un qui ose dire : quelque chose n'est pas clair, quelque chose ne tient pas, quelque chose mérite d'être regardé avant d'être amplifié.*

Ce livre commence là. Dans ce bureau. Devant cet ordinateur. Avec cet enfant qui refuse de se laisser convaincre que le problème est forcément de son côté.

Et qui, des années plus tard, a décidé de bâtir un système pour que personne d'autre n'ait à attendre aussi longtemps avant d'être cru.

---

PROLOGUE

# La question qui atout déclenché

> *L'Eureka ToTAL n'est pas arrivé dans un moment de calme. Il est arrivé devant le désordre ordinaire d'une organisation logicielle moderne. Et il a pris la forme d'une question simple : et si nos grands concepts avaient eux aussi besoin d'assurance qualité ?*

## Le désordre ordinaire

Il y a des organisations qui ont l'air de tout faire bien. Elles ont adopté les frameworks agiles. Elles ont investi dans l'automatisation. Elles ont des cérémonies régulières, des métriques soigneusement collectées, des environnements bien entretenus. Leurs équipes sont compétentes. Leurs gestionnaires sont de bonne foi. Leurs livraisons sont régulières.

Et pourtant. Et pourtant, il y a des tensions qui reviennent. Des reprises qui se répètent. Des livraisons qui arrivent à destination, mais qui nécessitent encore un peu d'héroïsme pour tenir leur promesse. Des rétrospectives où les mêmes thèmes reviennent, reformulés légèrement, mais fondamentalement identiques d'une itération à l'autre.

Ce n'est pas de la mauvaise volonté. Ce n'est pas de la négligence. Ce n'est pas un manque de compétence. Ce sont des personnes qui travaillent fort dans un système qui produit, malgré tous leurs efforts, un certain niveau de résistance irréductible.

C'est dans ce contexte précis — face à ce type d'organisation, face à ce type de résistance — que la question fondatrice de ToTAL est apparue.

## L'Eureka ToTAL

Les symptômes étaient familiers. Les besoins parfois mal compris, reformulés en cours de route, définis plus tard qu'ils n'auraient dû l'être. Les priorités changeantes, non pas parce que le marché changeait vraiment, mais parce que la conversation sur les priorités n'avait jamais été vraiment close. Les tests qui arrivaient trop tard dans le cycle pour permettre une correction confortable. Les livraisons (releases) qui demandaient une dernière montée en adrénaline collective. Et les tableaux de bord qui affichaient du vert sans que ce vert soit relié à une décision de maîtrise.

Le réflexe naturel — le réflexe de l'industrie depuis plusieurs décennies — est de chercher un meilleur processus. Un meilleur cadre. Un meilleur outil. Et souvent, cela aide. L'introduction de bonnes pratiques agiles, de DevOps, de tests automatisés, de chaînes de livraison (pipelines) robustes : ces investissements produisent des résultats réels.

Mais dans un certain nombre de cas, même après ces investissements, la résistance subsiste. Elle est plus polie. Elle est mieux habillée. Elle s'exprime dans un meilleur vocabulaire. Mais elle est là.

Et c'est là que la question est apparue. Pas comme une révélation soudaine, mais comme une évidence progressive qui prend soudainement une forme claire :

> *Et si le problème n'était pas seulement les processus ou les outils ? Et si nos grands concepts eux-mêmes — les mots que nous utilisons pour travailler ensemble — avaient besoin d'assurance qualité ?*

## Les mots qui semblent communs mais ne guident pas les mêmes décisions

Pensons à ce qui se passe réellement dans une organisation logicielle moyenne. Les personnes utilisent les mêmes mots. Valeur. Qualité. Produit. Test. Terminé (done). Livraison (release). Stratégie. Agile. DevOps. Transformation. Intelligence artificielle.

Ces mots circulent dans les réunions, dans les courriels, dans les présentations. Ils donnent l'impression d'un langage partagé. Ils créent l'apparence d'un alignement. Et dans cette apparence, quelque chose de subtil mais de puissant se cache : le fait que chaque personne autour de la table donne à ces mots un contenu légèrement différent.

Pour le directeur produit, « valeur » signifie l'impact mesurable sur les indicateurs d'affaires. Pour le développeur, « valeur » signifie un code propre, maintenable, extensible. Pour l'utilisateur final, « valeur » signifie que le logiciel fait ce qu'il m'a promis, sans friction, sans surprise. Pour le responsable qualité (QA), « valeur » signifie que ce qui est livré correspond à ce qui a été convenu, sans zones grises.

Ces quatre définitions ne sont pas incompatibles. Mais elles ne sont pas identiques. Et dans l'espace entre elles, se glissent les reprises, les malentendus, les livraisons qui déçoivent, et les rétrospectives qui circulent en boucle.

Il en va de même pour « qualité ». Pour « terminé (done) ». Pour « test ». Pour « Agile ». Chaque concept fondateur du travail logiciel porte en lui une

définition implicite qui varie d'une personne à l'autre, d'une équipe à l'autre, d'une organisation à l'autre. Et cette variation reste rarement visible tant que le système ne se fissure pas.

## La révélation : l'assurance qualité des concepts

L'assurance qualité (AQ) appliquée à un produit logiciel cherche à s'assurer que le produit correspond à ce qui a été défini, que ses comportements sont prévisibles, que ses défauts sont identifiés avant qu'ils n'atteignent l'utilisateur.

Mais qui assure la qualité des définitions elles-mêmes ? Qui vérifie que le mot « terminé (done) » recouvre bien la même réalité pour le développeur qui ferme le billet de travail (ticket) et pour l'utilisateur qui recevra la fonctionnalité ? Qui s'assure que « valeur » n'est pas un terme creux que chacun interprète à sa convenance ? Qui challenge la définition de « qualité » avant que le produit soit conçu, plutôt qu'après qu'il ait déçu ?

Personne. Ou presque. Et c'est précisément là que ToTAL entre.

L'assurance qualité des concepts — la AQ des concepts — est l'idée centrale, fondatrice et organisatrice de ToTAL. Avant d'assurer la qualité de ce que nous construisons, nous devons apprendre à assurer la qualité de ce avec quoi nous construisons.

Les outils avec lesquels nous construisons un produit logiciel ne sont pas seulement des logiciels et des serveurs. Ce sont aussi des mots. Des définitions. Des conventions implicites. Des concepts partagés — ou supposément partagés. Et ces concepts, comme tout autre composant d'un système complexe, peuvent être flous, incohérents, mal définis, ou contradictoires selon le contexte.

Lorsqu'un concept fondateur est flou, tout ce que l'on construit sur ce concept porte en lui ce flou. On peut l'ignorer pendant longtemps. On peut même

livrer malgré lui. Mais à un moment ou à un autre — dans une livraison importante, dans une crise, dans une transformation majeure — ce flou se révèle. Et c'est toujours plus coûteux à ce stade qu'il ne l'aurait été au début.

## Avant d'assurer la qualité de ce que nous construisons

Cette phrase n'est pas qu'une formule. Elle est un changement de regard.

Elle invite à reculer d'un pas avant de commencer. Pas pour ralentir le travail. Pas pour ajouter une couche de bureaucratie à un système déjà surchargé de réunions et de processus. Mais pour s'assurer que la conversation de départ — sur ce que nous construisons, pour qui, selon quels critères, avec quelle définition de la réussite — est une vraie conversation. Pas un monologue déguisé en réunion. Pas un alignement de façade. Pas un document de spécifications que personne n'a vraiment lu.

Une vraie conversation. Dans laquelle les mots que nous utilisons sont suffisamment clairs pour guider des décisions coherentes. Dans laquelle « terminé (done) » veut dire la même chose pour tout le monde. Dans laquelle « qualité » n'est pas un idéal abstrait mais un critère observable.

ToTAL n'invente pas cette exigence. Elle a toujours existé sous des formes diverses dans les meilleures pratiques de développement logiciel. Ce que ToTAL apporte, c'est une discipline structurée pour y répondre — de façon répétable, enseignable, et adaptable à des contextes très différents.

**■ À retenir**

- L'Eureka ToTAL est né d'une observation simple : dans les organisations qui ont déjà beaucoup de moyens, ce qui résiste encore souvent n'est pas un manque d'outils — c'est un manque de clarté sur les concepts fondateurs.

- Les mots que nous utilisons pour travailler ensemble — valeur, qualité, terminé (done), livraison (release), stratégie — semblent partagés mais ne guident pas toujours les mêmes décisions.
- Avant d'assurer la qualité de ce que nous construisons, nous devons apprendre à assurer la qualité de ce avec quoi nous construisons.

**▲ Symptômes à observer**

- Des rétrospectives où les mêmes thèmes reviennent, d'itération en itération, sans résolution durable.
- Des livraisons (releases) qui sont techniquement terminées mais qui nécessitent encore une validation informelle pour être vraiment considérées comme prêtes.
- Des conversations sur les priorités qui semblent closes mais qui se rouvrent à la moindre pression — signe que la définition de la valeur n'était pas aussi partagée qu'elle semblait l'être.

**◆ Questions de réflexion**

- Dans votre organisation, quelle est la définition officielle de « terminé (done) » ? Est-elle écrite ? Est-elle comprise de la même façon par le développeur, le responsable qualité, et l'utilisateur final ?
- Si vous demandiez à cinq collègues de définir « valeur » dans le contexte de votre produit, obtiendriez-vous cinq réponses identiques

— ou cinq variations révélatrices ?

- Quel concept dans votre travail quotidien est le plus souvent utilisé sans avoir été défini avec précision ? Quel impact cela a-t-il sur vos décisions ?

⚠️ **Signal de vigilance**

Si votre organisation utilise beaucoup le mot « alignement » sans jamais se donner le temps de vérifier si cet alignement est réel ou supposé, le flou conceptuel est probablement déjà au travail.

---

INTRODUCTION

# Pourquoi rien ne suffit,et pourquoi cela s'aggravemalgré tout

> *Les organisations logicielles modernes n'ont jamais eu autant de moyens pour mieux livrer. Et pourtant, beaucoup d'entre elles sentent que quelque chose résiste encore. Ce chapitre nomme ce paradoxe et propose une première clé pour le comprendre.*

## La promesse et la réalité

Depuis les années 1950, le monde du logiciel cherche à résoudre le même problème fondamental : livrer des systèmes qui fonctionnent, qui répondent aux besoins réels des personnes qui les utilisent, dans des délais et des coûts raisonnables, avec une qualité suffisante pour que la confiance soit au rendez-vous.

Pendant plusieurs décennies, la réponse a été la structure : des méthodes de gestion de projets, des cycles de développement formalisés, des processus de revue et d'approbation. Puis est venue l'agilité, avec sa promesse d'un développement plus itératif, plus collaboratif, plus proche des besoins réels. Puis DevOps, avec la promesse d'une livraison plus rapide, plus fiable, plus automatisée. Puis les plateformes infonuagiques (cloud), les chaînes de livraison (pipelines) continues, les tests automatisés, les tableaux de bord (dashboards) de métriques, les frameworks de mise à l'échelle agile.

Et maintenant l'intelligence artificielle : assistants de code, générateurs de tests, analyseurs de besoins, outils de revue automatisée. La promesse est immense. Les démonstrations sont convaincantes. Les premiers cas d'usage sont réels.

Sur papier, tout semble là. En pratique, beaucoup d'organisations sentent que quelque chose résiste encore.

## Les trois profils organisationnels

En observant des organisations logicielles de tailles, secteurs et maturités très différentes, trois profils émergent avec une régularité frappante :

**Le profil qui a accéléré plus vite qu'il n'a appris à se maîtriser.** Cette organisation a adopté les pratiques modernes avec enthousiasme. Elle livre vite. Elle itère vite. Elle déploie souvent. Mais la vitesse a devancé la compréhension. Les décisions sont prises à la hâte. Les apprentissages ne

s'accumulent pas. Les mêmes erreurs reviennent sous de nouvelles formes. La vitesse elle-même est devenue une forme de fuite devant la nécessité de ralentir pour comprendre.

**Le profil qui stagne.** Cette organisation a fait des efforts réels pour se moderniser. Elle a adopté Scrum, ou SAFe, ou une variante de DevOps. Les pratiques sont en place. Les rituels existent. Mais les résultats attendus ne sont pas tout à fait au rendez-vous. L'organisation stagne dans un plateau confortable : assez bien pour justifier de continuer, pas assez bien pour sentir qu'elle est vraiment maîtresse de son système de livraison.

**Le profil qui a décéléré.** Cette organisation a traversé une ou plusieurs transformations importantes. Elle en est ressortie fatiguée. La méfiance envers les nouvelles initiatives est réelle. Chaque nouveau projet de changement est regardé avec un scepticisme qui n'est pas irrationnel — il est l'accumulation de plusieurs déceptions. Le système s'est adaptatif au changement en développant une forme d'immunité.

Dans les trois cas, le problème central n'est pas la vitesse. C'est la maîtrise. Et la maîtrise ne s'acquiert pas en ajoutant des outils. Elle s'acquiert en développant une compréhension approfondie de ce que l'on fait, pourquoi on le fait, et comment on sait que cela fonctionne vraiment.

## Bienvenue chez ORIGINALIUM

Pour illustrer les concepts de ToTAL tout au long de cette collection, nous utiliserons une organisation fictive inspirée de l'expérience réelle : **ORIGINALIUM**.

ORIGINALIUM est une organisation de taille moyenne spécialisée dans le développement de logiciels. Elle n'est ni un géant technologique ni une petite startup. Elle est dans cette zone intermédiaire que beaucoup d'organisations occupent : assez grande pour avoir une complexité organisationnelle réelle,

assez petite pour que chaque décision importante soit encore humainement traçable.

Au cœur de notre histoire se trouve la **DREAM-TEAM**, une équipe de développement produit chargée d'un produit phare appelé **BEASTEST**.

La DREAM-TEAM est composée de :

- **Rania**, tech lead passionnée par la qualité systémique. Elle a une capacité rare à voir les connexions entre les décisions techniques et les résultats organisationnels. Elle est souvent celle qui pose la question inconfortable au bon moment.
- **Théo**, propriétaire du produit (product owner). Théo est l'interface entre les besoins des utilisateurs et la réalité de l'équipe. Il est empathique, organisé, mais parfois pris en étau entre des demandes concurrentes dont personne ne veut établir les priorités avec clarté.
- **Karim**, responsable de l'assurance qualité (QA). Karim est le défenseur silencieux de la cohérence. Il voit les fissures avant qu'elles ne deviennent des fractures. Il a appris, parfois douloureusement, que signaler un risque trop tôt peut vous valoir d'être considéré comme un frein plutôt qu'un allié.

BEASTEST est un outil optionnel qui peut faciliter et accélérer certaines pratiques de l'équipe — notamment le suivi, la visibilité, et la gestion des signaux qualité. Son utilisation n'est jamais imposée. L'équipe l'adopte progressivement, selon ses besoins.

Tout au long de cette collection, nous retrouverons Rania, Théo, Karim et leurs collègues dans des situations que beaucoup de praticiens reconnaîtront : des réunions qui auraient pu être des courriels, des courriels qui auraient dû être des conversations, des décisions prises sous pression, des signaux ignorés qui finissent par se venger.

## La chaîne fondatrice de ToTAL

ToTAL est organisé autour d'une conviction simple, mais dont les implications sont profondes :

> *La clarté produit la qualité. La qualité construit la confiance. La confiance permet l'excellence durable.*

Cette chaîne — clarté → qualité → confiance → excellence durable — est le fil conducteur de toute la collection. Chaque livre l'explore sous un angle différent. Chaque pratique, chaque outil, chaque concept de ToTAL peut être rattaché à l'un des maillons de cette chaîne.

L'Origine, ce premier livre, s'intéresse au maillon initial : la clarté. Pas la clarté comme un idéal. La clarté comme une pratique. La clarté comme un travail quotidien, délibéré, parfois difficile, que les organisations qui réussissent vraiment ont appris à faire systématiquement.

## L'intelligence artificielle comme révélateur

Un mot sur l'intelligence artificielle, parce qu'elle est désormais au centre de toutes les conversations sur l'avenir du développement logiciel.

L'IA change beaucoup de choses. Elle peut générer du code, des tests, des spécifications, des analyses. Elle peut accélérer des tâches qui prenaient des jours. Elle peut rendre accessibles des capacités qui étaient réservées à des spécialistes.

Mais l'IA, comme tout outil puissant, amplifie ce qui existe déjà. Si ce qui existe déjà est de la clarté — des besoins bien définis, des critères observables, des décisions bien comprises — l'IA peut amplifier cette clarté et la rendre encore plus efficace. Mais si ce qui existe déjà est du flou — des

besoins imprécis, des concepts mal définis, des décisions implicites — l'IA peut amplifier ce flou avec une vitesse et une élégance impressionnantes.

Un système flou accéléré n'est pas un meilleur système. C'est un système qui produit davantage d'erreurs convaincantes, plus rapidement.

**Respiration ToTAL**

*"Avant d'utiliser l'intelligence artificielle, demandez : qu'allons-nous amplifier exactement ?"*

Cette question — qu'allons-nous amplifier exactement ? — est une des questions ToTAL les plus importantes de notre époque. Pas parce qu'elle freine l'adoption de l'IA. Parce qu'elle garantit que cette adoption soit utile plutôt que simplement rapide.

**■ À retenir**

- Les organisations logicielles modernes ont plus d'outils, de pratiques et de frameworks que jamais — et pourtant, beaucoup d'entre elles sentent encore une résistance irréductible dans leur système de livraison.
- Trois profils organisationnels récurrents : accélérer sans maîtrise, stagner, et décélérer après trop de transformations. Dans les trois cas, le problème central est la maîtrise, pas la vitesse.
- L'intelligence artificielle amplifie ce qui existe déjà. Avant de l'adopter, la question ToTAL est : qu'allons-nous amplifier exactement ? Du flou amplifié n'est pas de la clarté accélérée.

### ▲ Symptômes à observer

- Votre organisation a adopté plusieurs pratiques modernes mais les résultats restent en deçà des attentes : l'investissement est réel, mais le retour est décevant.
- Les transformations se succèdent, chacune promettant ce que la précédente n'a pas livré. Le cynisme organisationnel augmente.
- L'adoption de l'IA génère de l'enthousiasme mais aussi une question sous-jacente : sommes-nous en train d'accélérer quelque chose que nous ne maîtrisions pas encore vraiment ?

### ◆ Questions de réflexion

- Dans quel profil organisationnel vous reconnaissez-vous le mieux — accélérer, stagner, ou décélérer ? Qu'est-ce que cela implique pour votre prochain mouvement ?
- Quelle est la dernière fois où votre organisation a ralenti délibérément — pas parce qu'elle était en crise, mais parce qu'elle voulait mieux comprendre avant de continuer ?
- Si votre organisation commençait à utiliser l'IA de façon intensive demain matin, qu'est-ce que vous amplifieriez exactement ? Seriez-vous satisfait de cette amplification ?

**⚠️ Signal de vigilance**

Si l'adoption d'un nouvel outil ou d'une nouvelle pratique dans votre organisation s'accompagne d'un enthousiasme collectif mais d'aucune conversation sérieuse sur ce que vous cherchez à maîtriser exactement, le cycle va probablement se répéter.

---

PARTIE 1

# Le chaos que les organisationsont appris à normaliser

**Note de partie**

*Cette première partie ne cherche pas à accuser les organisations. Elle ne cherche pas non plus à dramatiser des réalités que beaucoup d'équipes gèrent avec courage et compétence au quotidien. Elle cherche à nommer un phénomène que beaucoup vivent déjà : un chaos discret, devenu presque normal, qui se cache derrière les outils, les tableaux de bord, les rituels, les transformations et les livraisons régulières. Nommer ce phénomène est la première étape pour ne plus en être prisonnier.*

---

CHAPITRE 1

# Le paradoxe desorganisations modernes

> *Une équipe peut avoir tout ce qu'il faut en apparence — et pourtant manquer d'une chose essentielle. Ce chapitre explore ce paradoxe et propose la première question ToTAL pour le traverser.*

## Avoir tout — et manquer de maîtrise

Imaginez une équipe. Elle a un carnet de produit (backlog) bien alimenté, trié et estimé. Elle travaille en itérations (sprints) de deux semaines. Elle a des chaînes de livraison (pipelines) automatisées qui déploient dans plusieurs environnements. Elle a des tests automatisés qui couvrent une bonne portion de son code. Elle a un tableau de bord (dashboard) qui agrège ses métriques principales. Elle tient ses cérémonies agiles régulièrement. Elle a même commencé à intégrer des assistants d'intelligence artificielle dans son flux de travail.

Cette équipe a, sur le papier, tout ce que l'industrie recommande pour bien livrer un logiciel en 2025.

Et pourtant. Quand arrive le moment d'une décision difficile — livrer maintenant ou attendre une semaine de plus ? Prioriser cette fonctionnalité ou l'autre ? Marquer cet élément comme terminé (done) ou demander une révision ? — la confiance collective n'est pas toujours au rendez-vous. Il y a une hésitation. Un silence. Une demande d'avis supplémentaires. Une escalade vers une personne qui aura, elle aussi, une opinion mais pas nécessairement plus d'informations pertinentes.

C'est le paradoxe central des organisations modernes : jamais autant de moyens pour contrôler le travail logiciel, mais pas toujours plus de maîtrise réelle de ce travail.

La maîtrise — au sens ToTAL — n'est pas l'absence de problèmes. Les organisations les plus saines ont des problèmes. La maîtrise, c'est la capacité à voir ces problèmes clairement, à les traiter avec les bons outils, et à apprendre d'eux de façon systématique. La maîtrise, c'est que les décisions importantes soient prises avec une confiance qui repose sur des éléments réels — pas sur l'impression que le tableau de bord est vert ce matin.

> ▶️ **Scène ORIGINALIUM — La réunion de direction**
>
> C'est un mardi matin. La salle de conférence principale d'ORIGINALIUM est bien éclairée. Sur l'écran, le tableau de bord BEASTEST affiche des indicateurs soigneusement agrégés : couverture de tests à 78 %, taux de réussite des compilations (builds) à 94 %, vélocité de l'équipe stable sur les quatre dernières itérations (sprints), zéro incident critique en production depuis trois semaines.
>
> Le directeur des opérations, satisfait, fait défiler les métriques. « L'équipe performe bien, » dit-il. « On est dans les temps pour la prochaine livraison (release). »
>
> C'est à ce moment que Rania prend la parole. Elle le fait doucement, sans chercher à contredire. Elle pose une question.
>
> **Rania :** « Quel signal nous ferait réellement changer de décision aujourd'hui ? Sur ce tableau de bord — si un de ces chiffres baissait, lequel nous ferait dire : on reporte la livraison ? »
>
> Un silence s'installe. Pas un silence hostile. Un silence de réflexion. Quelqu'un commence à répondre, puis s'arrête. Quelqu'un d'autre regarde

> les métriques avec un regard légèrement différent.
>
> Le directeur reprend. « Bonne question. Je suppose que si la couverture de tests tombait en dessous de 70%... » Il s'arrête lui-même. « En fait, est-ce que 70 % versus 78 % change vraiment quelque chose pour nos utilisateurs ? »
>
> La conversation qui suit dure quarante minutes. Elle révèle ce que le tableau de bord, pourtant bien conçu, ne montrait pas : que plusieurs métriques existaient parce qu'elles étaient faciles à mesurer, pas parce qu'elles étaient reliées à une décision de maîtrise spécifique. La couverture de tests était mesurée, mais personne n'avait défini ce que signifiait un niveau de couverture suffisant pour ce produit spécifique, avec ces utilisateurs spécifiques, dans ce contexte spécifique.
>
> BEASTEST avait facilité la collecte de ces données. Mais la question de Rania avait révélé que la maîtrise n'était pas dans les données. Elle était dans la conversation que les données auraient dû alimenter — et qui n'avait jamais eu lieu.

## Confondre activité et maîtrise

Ce que la scène d'ORIGINALIUM illustre, c'est une confusion profondément ancrée dans la culture des organisations logicielles modernes : la confusion entre l'activité et la maîtrise.

L'activité est visible. Les billets de travail (tickets) bougent sur le tableau kanban. Les compilations (builds) réussissent dans la chaîne de livraison (pipeline). Les cérémonies agiles se tiennent à l'heure. Les métriques sont collectées, agrégées, affichées. Les itérations (sprints) se succèdent. Les livraisons (releases) arrivent.

La maîtrise est différente. La maîtrise, c'est savoir pourquoi les billets bougent, et si leur mouvement produit ce que nous voulions produire. C'est savoir si les compilations qui réussissent garantissent une qualité suffisante pour l'utilisateur. C'est savoir si les cérémonies servent vraiment les décisions qu'elles sont censées alimenter, ou si elles sont devenues des rituels confortables.

L'activité sans maîtrise peut être rassurante pendant un temps. Elle donne l'impression que le système fonctionne. Elle maintient une certaine cohésion sociale — tout le monde est occupé, donc le projet avance. Mais au moment où une décision difficile se présente, ou au moment où le système se confronte à une vraie pression, l'activité sans maîtrise se révèle pour ce qu'elle est : du mouvement, pas du progrès.

La première question ToTAL dans ce contexte n'est pas « quel outil manque ? ». C'est : « quelle maîtrise manque malgré les outils déjà présents ? »

Cette question change la direction du regard. Au lieu de chercher à compléter l'outillage, elle invite à regarder ce que l'outillage existant ne révèle pas. Au lieu de supposer que plus d'automatisation ou plus de métriques résoudra le problème, elle invite à se demander si le problème est dans les données ou dans la conversation que les données n'ont pas encore rendue possible.

> **▶️ Scène ORIGINALIUM — La réunion produit floue**
>
> Trois jours plus tard. Salle plus petite. Théo a convoqué une réunion pour discuter d'une demande récurrente des utilisateurs : « améliorer le suivi dans BEASTEST. »
>
> La réunion commence avec l'énergie caractéristique d'une équipe motivée. Karim propose d'abord d'ajouter un filtre par statut dans la vue principale. Théo pense qu'il faudrait peut-être repenser la structure des billets de

travail (tickets). Un développeur suggère une notification automatique en cas de retard. Une autre personne propose d'intégrer un assistant IA pour résumer l'état du suivi chaque matin.

Cinq solutions sur la table. Aucun problème clairement défini.

C'est Rania qui ralentit la réunion. Elle pose la question ToTAL :

**Rania :** « Avant d'aller plus loin — qu'est-ce qui est flou ici ? Le besoin ? La valeur ? Le critère de qualité ? »

La salle se fige. Puis, lentement, trois réponses différentes émergent :

**Théo :** « Les utilisateurs disent qu'ils n'arrivent pas à voir l'état global d'un projet sans ouvrir dix billets différents. »

**Karim :** « Moi, ce qui me préoccupe, c'est qu'on ne sait pas quand un risque qualité est en train de se matérialiser dans le carnet de produit (backlog). »

**Le développeur :** « Honnêtement, je pense qu'on perd du temps à chercher des informations qui devraient être visibles au premier coup d'œil. »

Trois problèmes différents. Un seul mot pour les nommer : « suivi ». Et sans la question de Rania, l'équipe aurait passé la réunion à concevoir des solutions pour un problème qui n'avait pas encore été défini avec suffisamment de précision pour que l'une de ces solutions soit vraiment juste.

BEASTEST peut faciliter et accélérer la réponse à chacun de ces trois problèmes — une fois qu'ils sont clairement distincts. Mais sans cette clarification, même le meilleur outil aurait produit trois fonctionnalités qui se superposent sans vraiment satisfaire personne.

## La première question ToTAL

Ces deux scènes — la réunion de direction et la réunion produit floue — illustrent la même vérité fondamentale : la question la plus importante n'est pas celle que l'on pose habituellement.

L'organisation habituellement pose : « Quel outil utilisons-nous ? Quelle pratique adoptons-nous ? Quelle métrique suivons-nous ? » Ces questions ne sont pas mauvaises. Elles sont insuffisantes.

La première question ToTAL est : **quelle maîtrise manque, malgré les outils déjà présents ?**

Cette question force un type d'honnêteté que les organisations trouvent souvent inconfortable. Parce qu'elle implique de regarder non pas ce qui manque dans l'outillage, mais ce qui manque dans la compréhension. Elle implique de regarder non pas ce que les métriques montrent, mais ce que les métriques ne montrent pas. Elle implique de regarder non pas les cérémonies qui se tiennent, mais les conversations qui ne se tiennent pas parce que personne ne sait encore comment les ouvrir.

Dans les livres suivants de la collection, cette question se déclinera en pratiques spécifiques, en techniques d'AQ conceptuelle, en modèles de conversation et de diagnostic. Mais elle commence ici. Elle commence par reconnaître que le paradoxe des organisations modernes est réel, qu'il n'est pas une honte, et qu'il est le point de départ d'un travail sérieux.

**■ À retenir**

- Une organisation peut avoir tous les outils recommandés par l'industrie et manquer encore de maîtrise réelle de son système de

livraison. Les outils ne produisent pas la maîtrise — ils la rendent possible si les conversations qui les accompagnent sont les bonnes.

- Confondre activité et maîtrise est l'une des erreurs les plus coûteuses et les moins visibles dans les organisations logicielles modernes. Le mouvement n'est pas le progrès.
- La première question ToTAL change la direction du regard : au lieu de « quel outil manque ? », elle demande « quelle maîtrise manque, malgré les outils déjà présents ? »

**▲ Symptômes à observer**

- Des métriques collectées régulièrement, mais dont personne ne sait précisément quel seuil déclencherait une décision différente.
- Des réunions où l'on génère des solutions avant que le problème soit clairement défini — et où la même solution est proposée pour des problèmes qui, en réalité, sont distincts.
- Un sentiment collectif que les cérémonies agiles se tiennent bien, mais que les décisions importantes se prennent toujours ailleurs, dans des conversations informelles.

**◆ Questions de réflexion**

- Dans votre équipe, quelle métrique, si elle changeait significativement demain matin, déclencherait une décision concrète ? Pouvez-vous nommer cette décision avec précision ?

- La dernière fois que votre équipe a discuté d'une « amélioration » à apporter — avez-vous défini le problème avant de proposer des solutions ? Combien de temps cela a-t-il pris ?
- Quelle maîtrise manque à votre équipe, malgré les outils qu'elle possède déjà ? Qu'est-ce que vous évitez de regarder directement ?

**⚠️ Signal de vigilance**

Si la réponse habituelle de votre organisation à un problème de livraison est d'ajouter un outil ou une métrique supplémentaire — sans d'abord clarifier ce que l'outil devrait rendre décidable — le paradoxe s'approfondit à chaque cycle.

---

CHAPITRE 2

# Accélérer, stagnerou décélérer

> *Les trois profils organisationnels ne sont pas des jugements. Ce sont des diagnostics. Et souvent, la même organisation vit les trois simultanément sur des parties différentes de son système de livraison.*

## Le système nerveux : accélérer sans maîtrise

Le premier profil organisationnel — celui qui accélère — est souvent le plus admiré de l'extérieur. Cette organisation livre souvent. Elle déploie en production avec une régularité impressionnante. Elle adopte les nouvelles pratiques rapidement. Elle est perçue comme une organisation « agile » au sens le plus positif du terme.

Mais regardons de plus près ce qui se passe à l'intérieur. La vitesse a été acquise, souvent, en réduisant les temps de réflexion. Les décisions sont prises rapidement parce qu'elles doivent l'être — le prochain sprint commence lundi. Les apprentissages sont partiels parce qu'il faut déjà passer à la prochaine itération (sprint). Les problèmes récurrents sont traités de façon ponctuelle parce qu'on n'a pas le temps de s'asseoir et de comprendre leur cause racine.

Ce profil développe ce que nous pouvons appeler un *système nerveux organisationnel* : hyperréactif, constamment en mouvement, produisant de l'énergie mais de moins en moins capable de distinguer le signal urgent du bruit de fond permanent. Les personnes sont occupées. Mais elles ne savent plus toujours pourquoi elles font ce qu'elles font.

Le paradoxe de ce profil est que sa vitesse même devient un obstacle à sa maîtrise. Il n'y a pas le temps de questionner. Il n'y a pas l'espace pour dire : attendons, quelque chose ne tient pas ici. Et sans cet espace, les erreurs de fond s'accumulent sous la surface, invisibles jusqu'au moment où elles deviennent des crises.

## Le système défensif : stagner

Le deuxième profil — celui qui stagne — a fait des investissements réels. Les pratiques sont en place. Les rituels existent. Les personnes sont formées. Et pourtant, les résultats ne décollent pas vraiment.

La stagnation est souvent confondue avec la stabilité. Mais ce ne sont pas la même chose. La stabilité est une maîtrise maintenue. La stagnation est un mouvement qui ne produit plus de progrès.

Ce profil a développé ce que nous pouvons appeler un *système défensif* : les pratiques sont adoptées non pas pour ce qu'elles produisent, mais pour éviter les critiques. Le carnet de produit (backlog) est maintenu parce que c'est ce que font les équipes agiles. Les rétrospectives se tiennent parce que c'est dans le cadre. Mais les insights de ces rétrospectives ne se transforment pas en changements durables. Les apprentissages ne s'accumulent pas. Le système stagne dans une conformité confortable.

La stagnation est particulièrement difficile à diagnostiquer parce qu'elle est invisible à ceux qui la vivent de l'intérieur. Le tableau de bord est raisonnablement vert. Les livraisons (releases) arrivent. Les équipes ne se plaignent pas — ou si elles le font, leurs plaintes ne trouvent pas d'écho dans les espaces où les décisions se prennent. Et l'organisation continue, dans une normalité qui ressemble à la santé mais qui en est, subtilement, très éloignée.

## Le système fatigué : décélérer

Le troisième profil — celui qui a décéléré — est le plus complexe à aborder, parce qu'il exige de reconnaître que des investissements importants n'ont pas produit les résultats attendus. Et cette reconnaissance est difficile pour les individus comme pour les organisations.

La décélération survient typiquement après une ou plusieurs transformations importantes. La transformation Agile qui s'est enlisée dans la bureaucratie des frameworks. La mise en œuvre DevOps qui a produit des chaînes de livraison (pipelines) impressionnantes mais n'a pas résolu les problèmes de communication entre développement et opérations. La migration infonuagique (cloud) qui a coûté plus cher et pris plus de temps que prévu. Le

programme de formation qui a créé du vocabulaire commun sans changer les comportements.

Après chaque déception, l'organisation développe ce que nous pouvons appeler un *système fatigué* : une méfiance envers les nouvelles initiatives, une inertie protectrice, une tendance à surévaluer les risques du changement et à sous-évaluer les coûts de l'immobilisme. La prochaine proposition de transformation est reçue avec un scepticisme parfois brutal : « Nous avons déjà essayé ça. »

## ORIGINALIUM : les trois profils simultanément

Ce qui rend ORIGINALIUM particulièrement représentative des organisations réelles, c'est qu'elle vit les trois profils en même temps — sur des parties différentes de son organisation et de son produit BEASTEST.

L'équipe de développement principale accélère. Elle livre vite, elle déploie souvent, elle adopte les nouveaux outils avec enthousiasme. Mais cette vitesse a créé une dette conceptuelle : certaines décisions fondamentales sur l'architecture du produit n'ont jamais vraiment été prises — elles ont été contournées, différées, ou résolues localement de façons incompatibles.

L'équipe de soutien (support) stagne. Elle a ses processus, ses billets de travail (tickets), ses escalades. Mais le lien entre ce que l'équipe de soutien (support) voit dans les incidents et ce que l'équipe de développement inclut dans son carnet de produit (backlog) est ténu. Les apprentissages issus des incidents de production ne se transforment pas systématiquement en améliorations préventives.

L'équipe de direction, elle, a décéléré. Elle a vu plusieurs initiatives de transformation ne pas livrer ce qu'elles promettaient. Elle est prudente, pour de bonnes raisons. Mais cette prudence se manifeste parfois comme un refus

de donner à de nouvelles approches l'espace nécessaire pour produire des résultats.

BEASTEST peut aider chacun de ces groupes à voir plus clairement sa situation — si l'équipe choisit de l'utiliser pour cela. Mais aucun outil ne peut forcer une organisation à avoir les conversations qu'elle évite.

## Le faux remède : ajouter des ressources au mauvais goulot

Face à l'un ou l'autre de ces trois profils, le réflexe organisationnel le plus courant est d'ajouter des ressources. Plus de développeurs pour accélérer. Plus de gestionnaires pour coordonner. Plus de processus pour structurer. Plus d'outils pour automatiser.

Ce réflexe est naturel. Il est visible. Il est politiquement acceptable — il montre que l'organisation prend le problème au sérieux. Mais il est souvent contre-productif, pour une raison précise que Frederick Brooks a décrite il y a plusieurs décennies et qui reste aussi vraie aujourd'hui : ajouter des personnes à un projet en retard le retarde davantage.

Pourquoi ? Parce que chaque nouvelle personne ajoutée à un système complexe augmente d'abord le coût de coordination avant d'augmenter la capacité de production. Elle a besoin d'être embarquée, formée, intégrée. Elle a besoin de comprendre le contexte, les décisions passées, les conventions existantes. Et pendant ce temps, les personnes déjà en place consacrent une partie de leur énergie à cet embarquement plutôt qu'à la production.

Mais il y a une raison plus profonde encore. Ajouter des ressources au mauvais goulot n'aide pas. Si le goulot est la clarté conceptuelle — si ce qui ralentit l'organisation est une définition floue de la valeur, un critère de terminé (done) incohérent, une priorisation qui change à chaque réunion — alors ajouter des développeurs produit davantage de code dans la mauvaise

direction, davantage de tests qui valident les mauvaises choses, davantage de livraisons (releases) qui déçoivent les utilisateurs.

Plus de capacité dans un système flou produit davantage de flou, plus vite.

**✎ Carnet de terrain — Karim, responsable AQ**

*« Dans une organisation où nous avions beaucoup d'outils et peu de clarté sur ce que nous construisions exactement, la direction a décidé d'ajouter trois développeurs à l'équipe. La logique était simple : plus de mains, plus de code, plus de fonctionnalités.*

*Ce qui s'est passé en réalité : les réunions ont doublé. Pas parce que nous étions inefficaces. Mais parce que trois nouvelles personnes avaient besoin de comprendre un contexte que nous n'avions jamais pris le temps de vraiment documenter. Et en essayant de l'expliquer, nous avons réalisé que nous-mêmes n'étions pas toujours d'accord sur certaines décisions fondamentales.*

*Les reprises ont aussi doublé. Parce que les trois nouveaux développeurs, en l'absence d'une définition claire de terminé (done), produisaient du code qui correspondait à leur interprétation du carnet de produit (backlog) — une interprétation différente de la mienne et de celle de Théo.*

*Ce n'est qu'après une réunion longue et parfois inconfortable, où nous avons pris le temps de définir ensemble ce que "terminé" voulait dire pour chaque type de billet de travail (ticket), que la vitesse est revenue — sans ajouter personne. Les trois nouveaux développeurs étaient toujours là. Mais maintenant, ils travaillaient dans la même direction que nous. »*

## La vraie question de maîtrise

Face aux trois profils — accélérer, stagner, décélérer — la vraie question à poser n'est pas : « comment aller plus vite ? ». C'est : **notre problème est-il la lenteur, la vitesse, ou l'absence de clarté sur ce que cette vitesse devrait rendre meilleur ?**

Cette question est différente de toutes celles que les organisations posent habituellement. Elle ne cherche pas à optimiser le système existant. Elle cherche à comprendre si le système est orienté vers la bonne direction avant de l'optimiser.

Dans les organisations qui accélèrent sans maîtrise, la réponse est presque toujours : le problème n'est pas la lenteur. Le problème est l'absence de clarté sur ce que la vitesse devrait produire.

Dans les organisations qui stagnent, la réponse est souvent : le problème n'est pas le manque de pratiques. Le problème est que les pratiques existantes ne sont pas connectées à une intention de maîtrise claire.

Dans les organisations qui ont décéléré, la réponse est parfois : le problème n'est pas le changement lui-même. Le problème est que les changements passés ont été imposés sans suffisamment de clarté sur ce qu'ils devaient rendre possible.

Dans les trois cas, la clarté — la AQ des concepts — est le point de départ.

**■ À retenir**

- Les trois profils organisationnels — accélérer sans maîtrise, stagner dans la conformité, décélérer après trop de déceptions — ne sont pas des jugements. Ce sont des diagnostics qui permettent d'identifier le

bon levier d'intervention.

- Ajouter des ressources au mauvais goulot aggrave souvent le problème qu'il cherche à résoudre : davantage de coordination, davantage de reprises, davantage de désalignement produit à plus grande vitesse.
- La vraie question de maîtrise n'est pas « comment aller plus vite ? » mais « notre problème est-il la lenteur, la vitesse, ou l'absence de clarté sur ce que cette vitesse devrait rendre meilleur ? »

**▲ Symptômes à observer**

- L'ajout de personnes à l'équipe a produit davantage de réunions et de reprises, mais pas davantage de fonctionnalités livrées avec confiance.
- Les nouvelles pratiques adoptées sont bien exécutées mais ne sont pas connectées à des indicateurs de maîtrise clairs — on fait le mouvement sans savoir exactement ce qu'il est censé produire.
- La direction exprime un scepticisme croissant envers toute nouvelle initiative de transformation — non pas parce qu'elle résiste au changement, mais parce qu'elle a vu trop de changements ne pas tenir leurs promesses.

**◆ Questions de réflexion**

- Dans quel profil votre organisation se reconnaît-elle le mieux en ce

moment : accélération, stagnation, ou décélération ? Est-ce le même profil dans toutes les équipes, ou des profils différents coexistent-ils ?

- La dernière fois que votre organisation a ajouté des ressources pour résoudre un problème de livraison — est-ce que cela a résolu le problème, ou a-t-il simplement pris une autre forme ?
- Si vous pouviez nommer le « mauvais goulot » dans votre organisation aujourd'hui — celui sur lequel vous ajoutez des ressources sans résultat — à quoi ressemblerait-il ? Est-ce un problème de capacité ou un problème de clarté ?

**⚠️ Signal de vigilance**

Si l'ajout de ressources dans votre organisation augmente surtout les réunions, les reprises et la coordination, le problème n'était probablement pas la capacité. C'était la clarté.

---

CHAPITRE 3

# Le chaos n'est pas seulementde la mauvaise volonté

> *Dans les organisations logicielles, la tentation de chercher un coupable est*

> *forte et compréhensible. Mais réduire le chaos à la mauvaise volonté empêche de voir le système. Et c'est le système qui doit changer.*

## La tentation du coupable

Quand une livraison (release) déçoit, quand un projet déraille, quand les mêmes erreurs reviennent en boucle, la réaction la plus humaine est de chercher une cause. Et souvent, cette cause prend la forme d'une personne. Le gestionnaire qui n'a pas bien défini les exigences. L'équipe de développement qui a livré quelque chose qui ne correspondait pas aux attentes. Le responsable qualité (QA) qui n'a pas testé suffisamment. Le chef de produit qui a changé les priorités sans préavis.

Cette réaction est compréhensible. Elle est même parfois partiellement juste. Certaines personnes prennent de mauvaises décisions. Certaines équipes travaillent en dessous de leur potentiel. Certains gestionnaires créent des conditions qui nuisent à la performance collective.

Mais réduire le chaos à la mauvaise volonté — ou même aux erreurs individuelles — empêche le plus souvent de voir ce qui est réellement en jeu : un système dont les règles implicites, les définitions floues et les structures d'incitation créent des conditions dans lesquelles les conflits et les reprises sont presque inévitables, indépendamment de la bonne volonté de chacun.

ToTAL adopte une posture radicalement différente : au lieu de commencer par demander « qui a fait l'erreur ? », elle demande « qu'est-ce qui, dans le système, a rendu cette erreur probable ? »

## Quand chacun agit à partir d'une définition différente

Voici un scénario que beaucoup de praticiens reconnaîtront :

Le développeur livre une fonctionnalité. Pour lui, cette fonctionnalité est terminée (done) : le code fonctionne, les tests unitaires passent, la compilation (build) réussit. Le responsable qualité (QA) reçoit la fonctionnalité et constate qu'elle ne correspond pas aux critères d'acceptation — mais ces critères n'étaient jamais écrits avec suffisamment de précision. Le chef de produit regarde le résultat et voit que la valeur promise n'est pas là — mais cette valeur n'avait jamais été définie avec des critères observables. L'utilisateur final teste la fonctionnalité et la trouve confuse — mais ses attentes n'avaient jamais été validées au-delà d'une phrase dans un courriel.

Qui a fait l'erreur ? Tout le monde, et personne. Chaque personne a agi de façon cohérente avec sa définition du travail bien fait. Le problème n'est pas la bonne volonté. Le problème est que quatre définitions différentes de « terminé (done) » et de « valeur » coexistaient dans le même système, invisibles à tous, jusqu'au moment de la livraison.

Ce type de conflit — déclenché non par la mauvaise volonté mais par des définitions implicites différentes — est l'un des plus courants et des plus dommageables dans les organisations logicielles. Il est coûteux en reprises. Il est épuisant pour les équipes. Il est destructeur pour la confiance entre les groupes. Et il revient, cycle après cycle, tant que les définitions fondamentales n'ont pas été rendues explicites et partagées.

## Le chaos comme symptôme de système

Le chaos — les reprises, les conflits, les livraisons (releases) qui déçoivent, les rétrospectives qui tournent en rond — est rarement la cause du problème. Il est presque toujours un symptôme. Un symptôme d'un système dont certains éléments fondateurs n'ont pas été clairement définis.

Cette distinction — entre le chaos comme cause et le chaos comme symptôme — est l'une des plus importantes de toute la discipline ToTAL. Parce qu'elle change complètement la nature de l'intervention.

Si le chaos est une cause, on cherche à l'éliminer directement : on impose plus de structure, on ajoute plus de contrôles, on renforce les processus existants. Si le chaos est un symptôme, on cherche à comprendre ce qu'il révèle : quelle définition est floue ? Quelle décision n'a jamais vraiment été prise ? Quelle convention implicite crée des attentes contradictoires ?

La réponse à un symptôme n'est pas de le supprimer. C'est de comprendre ce qu'il dit sur la santé du système sous-jacent.

## Le blâme ferme les boucles d'apprentissage

Il y a une conséquence organisationnelle du blâme qui est souvent sous-estimée : le blâme ferme les boucles d'apprentissage.

Dans un système où les erreurs sont immédiatement attribuées à des individus, les individus apprennent à éviter d'être vus en train de faire des erreurs. Ils apprennent à minimiser les problèmes, à les résoudre en silence avant qu'ils ne deviennent visibles, à ne pas signaler les signaux d'alerte précoces par peur d'être associés au problème.

Le résultat est que les problèmes deviennent invisibles jusqu'au moment où ils sont trop gros pour être ignorés. Les signaux précoces — les petits bugs, les flous conceptuels, les tensions dans l'équipe — sont supprimés avant d'avoir pu être traités. Et quand le problème éclate finalement, il est beaucoup plus coûteux à résoudre qu'il ne l'aurait été s'il avait été identifié et nommé plus tôt.

ToTAL propose une posture alternative : transformer le jugement en diagnostic, et le diagnostic en apprentissage. Non pas parce que cette posture est plus douce — elle exige souvent davantage de courage et

d'honnêteté que le blâme. Mais parce qu'elle est plus efficace. Parce qu'elle maintient ouvertes les boucles d'apprentissage qui permettent au système de s'améliorer réellement, plutôt que de s'améliorer en apparence.

## Carnet de terrain — l'expérience d'un système qui se dérègle

> ✎ **Carnet de terrain — expérience de terrain**
>
> *J'ai vu des initiatives presque prêtes à réussir. Des équipes motivées, des visions qui commençaient à se préciser, des pratiques qui commençaient à se relier en quelque chose de cohérent. Il y avait de la lumière dans ces projets. Quelque chose qui ressemblait à un début de maîtrise.*
>
> *Puis le système se déréglait. Pas toujours de façon spectaculaire. Souvent de façon discrète, presque chirurgicale. Un doute semé dans la bonne oreille. Une conversation déplacée à la mauvaise réunion. Un rapport rédigé avec les chiffres vrais mais le cadrage faux. La confiance qui se fissure non pas parce qu'une erreur a été faite, mais parce que quelqu'un a décidé que cette fissure était utile à ses objectifs.*
>
> *J'ai appris quelque chose d'important de ces expériences : la violence organisationnelle la plus dommageable n'est pas celle qui fait du bruit. C'est celle qui reste invisible, qui se glisse dans les espaces entre les réunions, qui se cache derrière la politesse des courriels formels et la courtoisie des présentations. C'est la violence qui fait douter ceux qui voient juste de la validité de ce qu'ils voient. Qui transforme un signal réel en une question de perception.*
>
> *Ce genre de dérive existe dans les systèmes humains, même dans des organisations qui affichent de belles valeurs. Même dans des organisations qui ont tous les processus corrects, tous les rituels appropriés, tous les outils recommandés. La beauté des valeurs affichées ne garantit pas la*

*santé du système sous-jacent.*

*ToTAL peut réduire l'espace où cette violence se cache. Pas en l'éliminant — aucun système ne peut éliminer la dimension politique du travail humain collectif. Mais en rendant les concepts plus clairs, les décisions plus traçables, les apprentissages plus visibles. En créant des conditions où il est plus difficile de faire passer du flou organisationnel pour de la complexité légitime. En rendant plus coûteux le fait de semer le doute sur quelqu'un qui voit correctement.*

*La qualité d'un produit logiciel dépend de beaucoup de facteurs techniques. Mais elle dépend aussi — et peut-être d'abord — de la qualité du système humain dans lequel ce produit est conçu. Un système humain où les personnes peuvent dire la vérité sans être détruites pour l'avoir dite. Un système où voir un problème est un acte de courage reconnu, pas un acte d'accusation puni. Un système où le signal précoce est bienvenu, parce que le signal précoce est ce qui évite les crises tardives.*

## Ce que ce carnet révèle sur les organisations

L'expérience décrite dans ce carnet de terrain n'est pas une exception. Elle est, à des degrés divers, une réalité vécue dans de nombreuses organisations logicielles. La politique existe. Les intérêts conflictuels existent. La tentation de protéger sa position aux dépens de la vérité collective existe.

ToTAL ne prétend pas pouvoir éliminer cette dimension humaine du travail organisationnel. Ce serait naïf et présomptueux. Mais ToTAL prétend — et la pratique le confirme — que la clarté conceptuelle réduit l'espace où cette dimension peut devenir destructrice.

Quand la définition de terminé (done) est claire et partagée, il est plus difficile de prétendre qu'une fonctionnalité est terminée quand elle ne l'est pas.

Quand les critères de valeur sont explicites, il est plus difficile de prétendre qu'une livraison (release) a livré de la valeur quand elle ne l'a pas fait. Quand les décisions sont tracées et leurs raisons documentées, il est plus difficile de réécrire l'histoire à la faveur de l'analyse rétrospective.

La clarté conceptuelle n'est pas seulement une question d'efficacité organisationnelle. C'est aussi une question d'équité. Elle crée des conditions où ceux qui voient correctement peuvent être crus. Elle crée des conditions où signaler un problème est un acte de contribution, pas un acte de dénonciation.

## La posture ToTAL : du jugement au diagnostic

La posture ToTAL face au chaos ne commence jamais par le jugement. Elle commence par le diagnostic.

Le diagnostic demande : qu'est-ce qui, dans ce système, a rendu ce chaos probable ? Quelles définitions étaient insuffisamment claires ? Quelles décisions n'ont jamais vraiment été prises et ont été traitées comme si elles l'avaient été ? Quelles conventions implicites ont créé des attentes contradictoires que personne n'avait explicitement alignées ?

Le diagnostic ouvre une conversation différente de celle que le jugement permet. Le jugement produit de la défensive, du silence, ou de la contre-accusation. Le diagnostic produit de la curiosité, de l'apprentissage, et — progressivement — des systèmes qui s'améliorent durablement.

Cela ne signifie pas que ToTAL évite les conversations difficiles. Au contraire. La posture ToTAL exige souvent de nommer des choses inconfortables. Mais elle les nomme en tant que symptômes d'un système, pas en tant que preuves de la faute d'une personne. Et cette distinction change tout.

Dans les organisations où cette posture est adoptée durablement, quelque chose de remarquable se produit : les personnes commencent à partager les

signaux précoces plus facilement. Parce qu'elles savent que le signal sera reçu comme une contribution, pas comme une accusation. Parce qu'elles ont vu, dans leur propre expérience, que nommer le problème tôt produit de meilleurs résultats que le résoudre en silence ou l'ignorer jusqu'à la crise.

C'est cela, en dernière analyse, la maîtrise systémique. Pas seulement des métriques vertes sur un tableau de bord. Pas seulement des livraisons régulières. Mais un système organisationnel où les signaux peuvent circuler librement, où les définitions sont suffisamment claires pour guider les décisions, et où apprendre est plus valorisé qu'avoir raison.

**■ À retenir**

- Le chaos dans les organisations logicielles est rarement une question de mauvaise volonté. Il est le plus souvent le symptôme d'un système dans lequel des définitions fondamentales — terminé (done), valeur, qualité — ne sont pas partagées explicitement.
- Le blâme ferme les boucles d'apprentissage. Il apprend aux individus à cacher les problèmes plutôt qu'à les signaler, transformant les signaux précoces en crises tardives.
- La posture ToTAL transforme le jugement en diagnostic : au lieu de demander « qui a fait l'erreur ? », elle demande « qu'est-ce qui, dans le système, a rendu cette erreur probable ? » Cette question change la nature de l'intervention — et ses résultats.

**▲ Symptômes à observer**

- Les mêmes types de conflits reviennent régulièrement entre les

mêmes groupes — développement, qualité, produit, direction — avec une régularité qui suggère un problème systémique plutôt qu'une incompatibilité personnelle.

- Les problèmes sont rarement signalés avant de devenir urgents. Les équipes résolvent les problèmes en silence plutôt que de les nommer, par peur d'être associées à ces problèmes.
- Les rétrospectives produisent des listes d'actions qui ne sont jamais vraiment mises en œuvre, parce que les causes racines identifiées sont des symptômes de causes plus profondes qui n'ont jamais été nommées.

**◆ Questions de réflexion**

- Dans votre organisation, quand un problème survient, quelle est la première question posée : « qu'est-ce qui s'est passé dans le système ? » ou « qui est responsable de ce qui s'est passé ? » Qu'est-ce que cette première question révèle sur votre culture organisationnelle ?
- Pensez à un conflit récurrent dans votre équipe. Si vous l'analysiez comme un symptôme de système plutôt que comme un problème de personne — qu'est-ce que vous verriez que vous ne voyez pas actuellement ?
- Est-ce que les personnes dans votre organisation signalent facilement les problèmes en émergence ? Ou attendez-vous souvent que les problèmes soient urgents pour en entendre parler ? Qu'est-ce

que cela dit sur les boucles d'apprentissage de votre système ?

**⚠️ Signal de vigilance**

Si dans votre organisation, dire « je vois un problème » ressemble à un risque plutôt qu'à une contribution, les boucles d'apprentissage sont déjà compromises — et les prochains problèmes seront plus gros que les précédents.

---

QUATRIÈME DE COUVERTURE — L1 PARTIE A

Il y a, dans chaque organisation logicielle, un moment précis où l'on réalise que tous les outils sont en place, tous les processus sont adoptés, toutes les cérémonies sont tenues — et pourtant, quelque chose résiste encore. Les livraisons arrivent mais déçoivent. Les rétrospectives tournent. Les transformations se succèdent sans vraiment atterrir. Ce moment n'est pas un échec. C'est un signal.

*« Avant le code, avant le test, avant l'outil, avant l'automatisation et avant l'IA, il y a quelqu'un qui ose dire : quelque chose n'est pas clair, quelque chose ne tient pas, quelque chose mérite d'être regardé avant d'être amplifié. »*

---

L'Origine est le premier livre d'une collection de vingt-et-un. Il est le pourquoi avant le comment. Il plante la graine d'une question fondatrice — et si nos grands concepts avaient eux aussi besoin d'assurance qualité ? — et montre comment

cette question, poursuivie avec rigueur et honnêteté, peut transformer la façon dont une organisation conçoit, livre et apprend.

L'auteur de ToTAL puise dans des décennies de pratique de terrain pour nommer ce que beaucoup d'équipes vivent sans encore l'avoir formulé : le chaos n'est pas de la mauvaise volonté, les outils n'achètent pas la maîtrise, et la clarté conceptuelle est le maillon le plus souvent négligé de la chaîne qui mène à l'excellence durable.

---

**21 livres. Une seule discipline : réaliser avec maîtrise.**

Collection ToTAL — Réconciliation (14 volumes) & Guides (7 volumes)

Images et couverture par Soléane Bélanger

**ToTAL — L'art de réaliser un produit logiciel avec maîtrise.**

---